

Pulyn — Documentação Técnica e Treinamento

Sep 30, 2026 · @Advantag

Visão geral

O Pulyn é uma plataforma de jogos com pulseiras NFC para eventos em buffets infantis: as crianças passam a pulseira em checkpoints, ganham pontos, e os pais acompanham tudo em tempo real pelo celular.

O sistema tem cinco partes:

Componente	Pasta	Tecnologia	Quem usa
Backend / API	backendPulyn	Node.js, Express, WebSocket (ws), JWT	Todos os outros componentes
Painel web	front-pulyn	React 18, TypeScript, Vite, Tailwind, Zustand	Admin, recepção, game master, telão
App Pulyn Family	pulyn_app	Flutter 3.47, Riverpod, go_router, Dio	Pais e responsáveis
Checkpoints	checkpoints	Firmware ESP32 + leitor RC522 (NFC)	Crianças (passam a pulseira)
Banco de dados	migrações em backendPulyn/migrations	PostgreSQL (migrado de SQL Server)	Backend

Três tipos de jogo são suportados: **Zone Conquest** (conquista de zonas), **Treasure Hunt** (caça ao tesouro, com cofre) e **Monster Hunt** (caça ao monstro, em times).

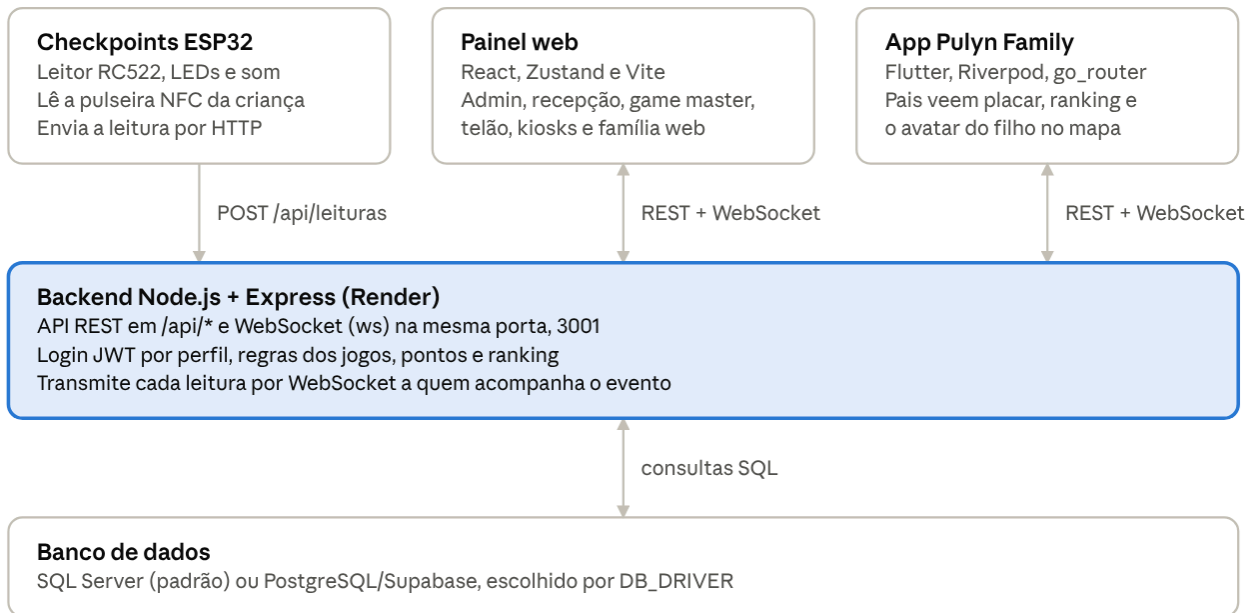
Ciclo de um evento

1. A recepção cadastra a criança e vincula a pulseira NFC.
2. O admin cria o evento, escolhe o jogo, os times e posiciona os checkpoints no mapa.
3. O evento passa de `scheduled` para `active` ; os pais recebem o convite da família e acompanham pelo app.

4. A criança passa a pulseira no checkpoint; o ESP32 envia `POST /api/leituras`; o backend calcula pontos e transmite `TERRITORY_CONQUERED` por WebSocket.
5. O telão, o painel e o app do pai atualizam placar, ranking e posição do avatar no mapa.
6. O evento passa para `completed` e o ranking final é exibido.

Arquitetura

Toda leitura de pulseira passa pelo backend, que atualiza painel e app



arquitetura do Pulyn · 3 clientes, backend e banco

O backend é o único ponto que fala com o banco; checkpoints, painel e app só conversam com ele. Uma leitura de pulseira entra por HTTP e volta como evento de WebSocket para todos que acompanham aquele evento.

Fluxo de uma leitura, passo a passo

1. O ESP32 lê o UID da pulseira e envia `POST /api/leituras` com o checkpoint e o UID.
2. O backend valida pulseira, checkpoint e modo do evento; aplica a regra do jogo; grava a leitura e atualiza `criancas.scores` e os pontos do time.
3. O backend transmite o evento com `global.broadcastToEvent(eventoId, ...)`, que entrega só aos clientes conectados àquele `evento_id`.
4. Painel e telão recarregam times, crianças, checkpoints e log de pontos; o app move o avatar e atualiza placar e ranking.
5. O ESP32 recebe a resposta e acende LED e toca som de sucesso ou erro.

Isolamento. Cada empresa (buffet) vê só os próprios dados: o JWT carrega `empresa_id`, as queries filtram por ele e o WebSocket recusa evento de outra empresa (exceto `master`). Quem acompanha o evento pelo app é sempre um responsável aprovado e vinculado à criança.

Backend e API

O backend é um único servidor Node.js (`backendPulyn/index.js`) que serve a API REST em `/api/*` e o WebSocket na mesma porta (`PORT` , padrão 3001). Em produção roda no Render (`https://backendpulyn.onrender.com`).

Banco de dados. `database.js` escolhe o driver por `DB_DRIVER: sqlserver` (padrão) ou `postgres` (Supabase, pool de até `PG_POOL_MAX` , padrão 10). As queries usam parâmetros nomeados (`@id`). O schema é criado e ajustado na subida do servidor pelas funções `ensure*Schema()` em `migrations/`; os scripts `npm run migrate` , `schema:postgres` e `data:migrate` cobrem a migração SQL Server para PostgreSQL.

Autenticação. `POST /api/auth/login` valida e-mail e senha na tabela `logins` e devolve um JWT de 24 horas com `id` , `email` , `empresa_id` e `role` . O token vai no cabeçalho `Authorization: Bearer <token>` . Os perfis válidos são:

Perfil (<code>role</code>)	Quem é	Tela após o login
<code>master</code>	Equipe Pulyn (todas as empresas)	<code>/master</code>
<code>admin</code>	Dono/gestor do buffet	<code>/admin</code>
<code>reception</code>	Recepção	<code>/reception</code>
<code>game_master</code>	Mestre do jogo	<code>/game-master</code>
<code>display</code>	Telão do evento	<code>/display</code>
<code>family</code>	Pai ou responsável (app)	<code>/family</code>
<code>kiosk</code>	Autoatendimento de cadastro	<code>/reception/kiosk</code>
<code>score_kiosk</code>	Autoatendimento de pontuação	<code>/score-kiosk</code>

Os perfis de kiosk só podem usar a API dedicada (`/api/kiosk` , `/api/score-kiosk`); qualquer outra rota responde 403. Os dados são isolados por empresa (`empresa_id`).

Rotas principais (prefixo `/api`):

Prefixo	Função
<code>/auth</code>	Login, logout e cadastro
<code>/clientes</code> , <code>/planos</code> , <code>/master</code> , <code>/analytics</code>	Gestão de empresas clientes, planos e métricas da equipe Pulyn
<code>/logins</code>	Usuários de cada empresa
<code>/eventos</code>	CRUD de eventos, crianças e times do evento, planta baixa, zonas, iniciar e parar jogo
<code>/brincadeiras</code> , <code>/times</code>	Jogos e times
<code>/criancas</code> , <code>/pulseiras</code>	Cadastro de crianças, vínculo de pulseira e QR Code
<code>/checkpoints</code>	Checkpoints do evento, heartbeat, configuração e território
<code>/leituras</code>	Leitura de pulseira vinda do ESP32 (o coração do jogo) e histórico
<code>/zone-conquest</code> , <code>/treasure</code> , <code>/monster</code>	Estado de cada tipo de jogo
<code>/ranking</code>	Ranking de crianças e de times por evento
<code>/familias</code> , <code>/family</code> , <code>/qrcode</code>	Convites, aprovação de vínculos e app das famílias
<code>/kiosk</code> , <code>/score-kiosk</code>	Terminais de autoatendimento
<code>/event-control</code>	Evento ativo da empresa
<code>/messages</code> , <code>/support</code> , <code>/logs</code> , <code>/settings</code> , <code>/monitoring</code>	Mensagens, suporte, logs, configurações e saúde das unidades

Rotas de depuração e manutenção ficam em `/api/debug/*` (iniciar e parar jogo, zerar pontuação, resetar territórios). A maioria exige `admin` , `game_master` ou `master` ; as destrutivas exigem `master` . Use `GET /api/test` como health check.

WebSocket. O cliente conecta em `ws://host:3001?evento_id=<id>` e envia o JWT no subprotocolo `pulyn-auth` . Com `scope=company` recebe o controle do evento da empresa. O servidor faz ping a cada 30 s e só aceita comandos `SET_MODE` e `COMMAND` de usuários autenticados que não sejam kiosk. Os modos válidos são `idle` , `checkin` , `bracelets` , `participants` e `game` .

Variáveis de ambiente (veja `.env.example`):

Variável	Uso
<code>DB_DRIVER</code>	<code>sqlserver</code> ou <code>postgres</code>
<code>DB_SERVER</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	SQL Server
<code>DATABASE_URL</code> / <code>SUPABASE_DB_URL</code> , <code>DB_SSL</code> , <code>PG_POOL_MAX</code>	PostgreSQL/Supabase
<code>DB_TIMEOUT</code>	Timeout de conexão em ms (padrão 30000)
<code>PORT</code> , <code>NODE_ENV</code>	Porta e ambiente
<code>JWT_SECRET</code>	Segredo do JWT
<code>FRONTEND_URL</code>	Base dos links de convite familiar

Painel web e app Pulyn Family

Painel web (`front-pulyn`)

É uma SPA React com `react-router-dom` . Cada perfil do backend tem sua área, protegida por `ProtectedRoute` com a lista de perfis permitidos. O estado global fica em um store Zustand (`store/mockData.ts` , apesar do nome ele carrega dados reais da API), e `services/api.ts` faz as chamadas HTTP.

Área	Rotas	Telas principais
Login	<code>/login</code>	Login único; redireciona pelo perfil
Admin (<code>admin</code>)	<code>/admin/*</code>	Dashboard, eventos (criar e editar), crianças, jogos, checkpoints e configuração, mapa, usuários, times, relatórios, sincronização, configurações
Recepção (<code>reception</code>)	<code>/reception/*</code>	Dashboard, check-in, participantes, pulseiras, famílias e vínculos familiares
Kiosk (<code>kiosk</code> , <code>score_kiosk</code>)	<code>/reception/kiosk</code> , <code>/score-kiosk</code>	Autocadastro e consulta de pontuação pela criança ou responsável

Área	Rotas	Telas principais
Game master (<code>game_master</code>)	<code>/game-master/*</code>	Controle do jogo, times, checkpoints, mensagens, ranking, configuração de zonas
Telão (<code>display</code>)	<code>/display/*</code>	Principal, ranking, mapa e standby
Master (<code>master</code>)	<code>/master/*</code>	Clientes, planos, monitoramento, logs, suporte, analytics
Família web (<code>family</code>)	<code>/family/*</code>	Início, localização, pontuação, conquistas, quiz, perfil, notificações; convite em <code>/family/invite/:token</code>

Tempo real. `useGameWebSocket` assina os eventos do evento atual; o telão recarrega times, crianças, checkpoints e log de pontos a cada evento de jogo (`TERRITORY_CONQUERED` , `TREASURE_PROGRESS` , `MONSTER_PROGRESS` e outros). Como reserva, `App.tsx` também faz polling a cada 10 s enquanto há evento selecionado. `useEventControl` mantém todas as telas da empresa no mesmo evento ativo, e `useNFCReader` e `useArduino` leem pulseiras em leitores conectados ao computador.

App Pulyn Family (`pulyn_app`)

App Flutter para pais e responsáveis, em Dart 3.13 e Flutter 3.47. Usa Riverpod para estado, `go_router` para rotas, Dio para HTTP, `web_socket_channel` para tempo real, `shared_preferences` para o token e `mobile_scanner` para ler o QR Code de vínculo com a criança.

Pasta em <code>lib/</code>	Conteúdo
<code>config/</code>	<code>api_config.dart</code> (ambientes e endpoints), <code>theme.dart</code>
<code>models/</code>	Modelos da família, criança, evento e rastreo de avatar
<code>providers/</code>	<code>auth_provider</code> , <code>events_provider</code> , <code>websocket_provider</code> e <code>index.dart</code> (crianças, placar, rastreo)
<code>services/</code>	<code>api_service</code> , <code>websocket_service</code> , <code>performance_service</code>

Pasta em lib/	Conteúdo
screens/	auth, home, child, event, family, onboarding, profile, ranking, qr_scan, settings
widgets/	Mapa do evento, heatmap, navegação inferior, painel de QR e componentes comuns

Rotas do app: /login, /register, /invite, /family/invite/:token, /onboarding, /home, /child/:id, /ranking, /profile, /notifications, /linked-children, /manage-children, /pre-event e /event-result. Na primeira execução o usuário vê o onboarding e depois cola o link do convite; nas seguintes, com sessão salva, vai direto para /home.

Na Home, event_phase_router.dart decide a tela pela situação do evento: scheduled mostra a pré-festa, active abre o mapa ao vivo e completed mostra o resultado. O ambiente da API (local, Render ou produção) é definido em ApiConfig.currentEnvironment; o padrão é Render.

Limitações atuais do app (detalhadas na seção de problemas conhecidos): a rota /event-map ainda não existe, a tela de notificações é um placeholder e a edição de perfil não grava.

Checkpoints, jogos e rastreo de avatares

Checkpoints (ESP32)

Cada checkpoint é um ESP32 com leitor RC522 (pinos RST=4, SS=5), fita de 7 LEDs (pino 21) e módulo de som DFPlayer (pinos 32 e 33). Há três firmwares em checkpoints/:

Firmware	Função	Endpoints que usa
pullynC2_modular	Checkpoint de jogo	GET /api/debug/checkpoint-mode, POST /api/checkpoints/:id/heartbeat, GET /api/checkpoints/:id/territory, POST /api/leituras
pullynReception	Leitor da recepção (vincula pulseira)	POST /api/leituras/reception

Firmware	Função	Endpoints que usa
<code>pullynScore</code>	Leitor de pontuação (autoatendimento)	<code>POST /api/auth/login</code> , <code>GET /api/event-control/active</code> , <code>POST /api/score-kiosk/readings</code>

Configuração em `config.h`: `PULYN_LAN_MODE` escolhe entre o servidor online (Render) e o servidor local da rede; `CHECKPOINT_ID` identifica o aparelho; o heartbeat sai a cada 60 s, o modo é consultado a cada 10 s, a mesma pulseira é ignorada por 5 s (`RFID_REPEAT_GUARD`) e o ESP32 tolera 3 falhas de API seguidas. Use sempre a pasta `pullynC2_modular` (a versão otimizada); `*_optimized.h` são cópias idênticas e podem ser ignorados. Sons: `01.mp3` para sucesso e ponto, `03.mp3` para erro. O monitor serial usa 115200 baud.

Regras dos jogos

Todos os jogos começam com a leitura da pulseira em `POST /api/leituras`. O backend valida a pulseira, o checkpoint e o modo do evento, grava a leitura, calcula os pontos e transmite o resultado por WebSocket ao evento.

Jogo (type)	Regras implementadas no backend
Zone Conquest, modo individual (zone)	10 pontos base, multiplicados por $1 + 0,01 \times$ checkpoints já lidos pela criança (resultado arredondado para baixo). Rerol o mesmo checkpoint só é permitido depois de ler 3 outros. Uma leitura já processada é rejeitada como duplicada.
Zone Conquest, modo equipe	A equipe conquista o território do checkpoint; não pode reconquistar o que já domina. Os pontos entram na criança e somam no total do time. O checkpoint reinicia após 90 s (<code>TEAM_CHECKPOINT_RESET_MS</code>).
Treasure Hunt (treasure_hunt)	Partida por rodadas com cofre; intervalo de 10 s entre turnos (<code>TREASURE_TURN_DELAY_MS</code>).
Monster Hunt (monster_hunt)	O monstro começa com 500 de vida. Checkpoint normal tira 10, checkpoint especial 30 e ataque especial 50. O cooldown padrão por checkpoint é 15 s, configurável de 1 a 120 s no cadastro do jogo.

Rastreo de avatares

Quando a criança lê a pulseira, o avatar dela vai para perto do checkpoint no mapa, no telão e no app dos pais. Para funcionar, o checkpoint precisa ter `map_x` e `map_y` preenchidos; sem eles, o backend registra `[RASTREIO] CRÍTICO` no log.

1. O backend inclui `mapX` e `mapY` no evento `TERRITORY_CONQUERED` (Zone Conquest, Treasure Hunt e Monster Hunt).
2. O cliente descobre o último checkpoint de cada criança pelo log de pontos e lê as coordenadas.
3. Crianças no mesmo checkpoint são distribuídas em posições fixas: três por linha, com deslocamentos horizontais de -35, 0 e +35 e linhas 50 abaixo uma da outra.
4. No app, o avatar se move em 800 ms com curva `easeInOutCubic`; coordenadas fora do mapa (350 × 280) ou ausentes caem no centro do mapa.

Eventos de WebSocket que o app usa: `TERRITORY_CONQUERED`, `SCORE_UPDATE`, `RANKING_UPDATE`, `EVENT_STARTED`, `EVENT_ENDED` e `NOTIFICATION`. O painel também trata `TERRITORY_RESET`, `TREASURE_PROGRESS`, `TREASURE_ROUND_COMPLETED`, `MONSTER_PROGRESS`, `MONSTER_SPECIAL_ATTACK`, `MONSTER_TEAM_DEFEATED` e `MONSTER_DEFEATED`.

Para diagnosticar um avatar que não se move: confira se o log do backend mostra `[RASTREIO] Enviando TERRITORY_CONQUERED`; se não mostra, faltam coordenadas no checkpoint. Se mostra, confira no app se chegou `[MAP] TERRITORY_CONQUERED RECEBIDO` (se não chegou, o WebSocket não conectou; revise `evento_id` e token) e depois `[ANIMATE_POS]`.

Operação, testes e problemas conhecidos

Como rodar cada parte

Parte	Comandos	Observações
Backend	<code>npm install</code> , copiar <code>.env.example</code> para <code>.env</code> , <code>npm run dev</code> (ou <code>npm start</code>)	Sobe na porta 3001 e cria as tabelas que faltam. <code>npm run test-connection</code> testa o banco.
Painel web	<code>npm install</code> , <code>npm run dev</code>	Vite em <code>http://localhost:5173</code> . Também: <code>npm run build</code> , <code>npm run lint</code> , <code>npm run typecheck</code> .

Parte	Comandos	Observações
App Flutter	<code>flutter pub get</code> , <code>flutter run</code>	Para emulador Android a API local é <code>10.0.2.2:3001</code> . Gerar ícones: <code>dart run flutter_launcher_icons</code> .
Checkpoint	Arduino IDE, placa ESP32 Dev Module, abrir <code>pullynC2_modular.ino</code>	Ajustar <code>CHECKPOINT_ID</code> , Wi-Fi e <code>PULYN_LAN_MODE</code> em <code>config.h</code> antes de gravar.

O backend roda no Render (<https://backendpulyn.onrender.com>). Hospedagem do painel web e processo de publicação do app nas lojas não estão documentados no repositório.

Testes (estado em 30/09/2026)

- **Flutter:** 72 testes; 69 passam e 3 falham (`test/entry_flow_test.dart`), porque procuram textos que a UI atual não tem ("Link ou código do convite", "Já tenho conta").
- **Backend:** `npm test` roda `test/*.test.js`; hoje existe só `familyInviteRules.test.js`. Os demais `test-*.js` da raiz são scripts manuais de verificação.
- **Painel web:** sem suíte de testes; só `lint` e `typecheck`.
- **flutter analyze**: 0 erros, 0 warnings e 5 avisos informativos.

Este levantamento não executou os testes do backend nem do painel.

Problemas conhecidos

Ordenados por prioridade. Nenhum deles foi corrigido; a lista vem do diagnóstico do app Flutter e da leitura do backend e do firmware.

Prioridade	Onde	Problema	Correção recomendada
Alta	<code>backendPulyn/routes/auth.js:34</code>	A senha é guardada e comparada em base64, que é reversível e não é hash.	Migrar para bcrypt ou argon2, com troca de senha gradual no login.
Alta	<code>utils/middleware.js:4</code> , <code>routes/auth.js:7</code> , <code>index.js:58</code>	Se <code>JWT_SECRET</code> não estiver definido, o servidor usa um segredo fixo no código.	Falhar na subida quando a variável faltar, e trocar o segredo em produção.
Alta	<code>pulyn_app/lib/screens/home/event_phase_router.dart:81</code>	Evento <code>active</code> navega para <code>/event-map</code> , rota que não existe no <code>main.dart</code> .	Registrar a rota do mapa ao vivo.
Média	<code>checkpoints/pullynC2_modular/config.h:16-17</code>	Senha do Wi-Fi gravada no código do firmware, que está no Git.	Mover para arquivo local fora do controle de versão e trocar a senha atual.

Prioridade	Onde	Problema	Correção recomendada
Média	<code>index.js:85-95</code>	O WebSocket aceita qualquer origem (<code>verifyClient</code> sempre verdadeiro).	Validar origem em produção.
Média	<code>pulyn_app/lib/main.dart (_buildRouter)</code>	O GoRouter é recriado a cada mudança de auth, o que zera o histórico de navegação.	Um único GoRouter com <code>redirect</code> e <code>refreshListenable</code> .
Média	<code>pulyn_app/lib/config/api_config.dart:18-20</code>	IP de rede local fixo no código.	Usar <code>--dart-define</code> .
Média	<code>pulyn_app/test/entry_flow_test.dart</code>	3 testes desatualizados.	Atualizar os textos esperados.
Baixa	<code>pulyn_app/lib/widgets/app_scaffold.dart:28</code>	<code>WillPopScope</code> depreciado.	Trocar por <code>PopScope</code> .
Baixa	<code>front-pulyn/src/App.tsx:216</code>	Comentário <code>//</code> dentro do JSX de rotas. O React Router ignora, mas é sujeira.	Trocar por <code>{/* ... */}</code> .
Baixa	<code>front-pulyn/src/pages/reception/</code>	Três versões do kiosk (<code>ReceptionKiosk</code> , <code>_otimizado</code> , <code>_backup_original</code>).	Confirmar qual está em uso e remover as outras.
Baixa	<code>index.js:108-137</code>	Verificação de checkpoints offline desativada por causar timeout no banco remoto.	Reativar com consulta leve e índice em <code>last_seen</code> .
Baixa	<code>pulyn_app/lib/providers/events_provider.dart</code> e <code>profile_screen.dart</code>	TODOs: quatro endpoints de eventos inexistentes, edição de perfil sem efeito, <code>/notifications</code> placeholder.	Implementar ou ocultar da interface.
Baixa	<code>pulyn_app/pubspec.yaml</code>	Dependências com versão major nova: Riverpod 3, go_router 18, permission_handler 13, intl 0.20.	Migrar uma por vez, com testes.

Divergências entre documentos antigos e o código

Os arquivos `.md` da raiz do repositório citam Socket.io e PostgreSQL como fixos. O código real usa o pacote `ws` (WebSocket puro) e o driver padrão é SQL Server; o PostgreSQL é ativado por `DB_DRIVER=postgres`, e o `database.js` converte a sintaxe T-SQL das queries para PostgreSQL. Este documento segue o código.

Treinamento

O treinamento tem cinco trilhas, uma por perfil. Cada pessoa faz a trilha do seu perfil mais o módulo comum. Todos os exercícios devem ser feitos em um **evento de teste**, nunca em um evento real, com pelo menos um checkpoint e duas pulseiras de teste.

Trilha	Público	Duração sugerida	Módulos
0. Comum	Todos	30 min	O que é o Pulyn, fluxo do evento, perfis e login
1. Recepção	Recepcionistas	1 h 30	Cadastro, pulseiras, famílias, kiosk
2. Game master	Recreacionistas	1 h 30	Escolher e iniciar o jogo, acompanhar, resolver problemas em campo
3. Admin do buffet	Gestores	2 h	Eventos, jogos, times, checkpoints, mapa, usuários, relatórios
4. Famílias	Pais (material para a recepção entregar)	15 min	Convite, vínculo com a criança, acompanhar pelo app
5. Técnica	Suporte e desenvolvedores	3 h	Instalação, deploy, checkpoints, diagnóstico e manutenção

Módulo 0 · Comum (todos)

Objetivo: entender o ciclo do evento e saber entrar no sistema.

1. Explicar em uma frase: a criança usa uma pulseira NFC, passa nos checkpoints, ganha pontos, e os pais acompanham pelo celular.
2. Percorrer o ciclo: cadastro, vínculo da pulseira, evento ativo, leituras, ranking, encerramento.
3. Entrar em `/login` com o e-mail e a senha do seu perfil; o sistema leva à área certa (`/admin`, `/reception`, `/game-master` ou `/display`).
4. Observar o evento atual: todas as telas da empresa seguem o mesmo evento ativo.

Verificação: cada participante diz em voz alta quais telas o seu perfil pode abrir e por que o perfil de outro colega abre telas diferentes.

Trilha 1 · Recepção

Objetivo: cadastrar crianças, vincular pulseiras e aprovar famílias sem erro.

1. Abrir o painel da recepção (`/reception`) e conferir o evento selecionado.

2. Em **Check-in** (/reception/checkin) e **Participantes** (/reception/participants), cadastrar a criança com nome, apelido e time.
3. Em **Pulseiras** (/reception/bracelets), vincular a pulseira à criança passando-a no leitor da recepção. Uma pulseira só pode estar vinculada a uma criança por vez.
4. Em caso de troca, desvincular a pulseira antiga da criança antes de vincular outra.
5. Gerar o QR Code da criança e entregá-lo ao responsável para o vínculo com o app.
6. Em **Famílias** (/reception/families) e **Vínculos** (/reception/family-links), aprovar ou rejeitar os pedidos pendentes. Conferir que o responsável é quem diz ser antes de aprovar.
7. Kiosk: explicar aos pais que o autocadastro (/reception/kiosk) e a consulta de pontos (/score-kiosk) são telas separadas, com login próprio.

Exercícios

- ☐ Cadastrar 2 crianças de teste em times diferentes.
- ☐ Vincular uma pulseira a cada uma e confirmar que aparece o vínculo.
- ☐ Trocar a pulseira de uma criança.
- ☐ Gerar o QR Code de uma criança e ler com o app de teste.
- ☐ Aprovar um vínculo familiar pendente e rejeitar outro.

Erros comuns: pulseira já vinculada a outra criança; criança cadastrada no evento errado; aprovar vínculo sem conferir o responsável.

Trilha 2 · Game master

Objetivo: conduzir uma partida do início ao fim e reagir a problemas.

1. Abrir /game-master e confirmar o evento e o jogo selecionado. Os jogos são Zone Conquest, Treasure Hunt e Monster Hunt (regras na seção de checkpoints).
2. Conferir os times (/game-master/teams) e os checkpoints (/game-master/checkpoints): todos devem aparecer online. Um checkpoint offline não enviou heartbeat nos últimos minutos.
3. No Zone Conquest, conferir a configuração das zonas (/game-master/zone-setup).
4. Iniciar o jogo no painel. O telão passa a mostrar a partida e os checkpoints mudam para o modo de jogo.
5. Durante a partida, acompanhar o ranking (/game-master/ranking) e enviar avisos pelas mensagens (/game-master/messages).
6. Se necessário, resetar um território ou zerar as pontuações; essas ações são irreversíveis na partida em curso.
7. Encerrar o jogo no painel e mostrar o ranking final no telão.

Exercícios

- ☐ Iniciar um Zone Conquest, passar duas pulseiras em checkpoints diferentes e ver a pontuação subir.
- ☐ Reler o mesmo checkpoint logo em seguida e explicar por que a leitura foi recusada (regra: só após 3 outros checkpoints, no modo individual).
- ☐ Desligar um checkpoint e ver o status mudar para offline; religar.
- ☐ Encerrar a partida e iniciar um Monster Hunt.

Erros comuns: iniciar o jogo com checkpoint offline; encerrar sem avisar o telão; resetar pontuação do evento errado.

Trilha 3 · Admin do buffet

Objetivo: montar um evento completo antes do dia da festa.

1. Criar o evento em **Eventos** (`/admin/events/new`): nome, data e jogo.
2. Cadastrar ou revisar os jogos em **Jogos** (`/admin/games`); no Monster Hunt, definir o cooldown de cada checkpoint (1 a 120 s).
3. Criar os times em **Times** (`/admin/teams`).
4. Cadastrar os checkpoints em **Checkpoints** (`/admin/checkpoints`) e ajustar cada um em `/admin/checkpoints/:id`. O `CHECKPOINT_ID` gravado no ESP32 deve ser igual ao cadastrado aqui.
5. No **Mapa** (`/admin/map`), carregar a planta baixa e posicionar cada checkpoint. Sem `map_x` e `map_y` o avatar não se move.
6. Criar os usuários da equipe em **Usuários** (`/admin/users`) com o perfil certo: recepção, game master, telão e kiosk.
7. Depois da festa, consultar **Relatórios** (`/admin/reports`) e **Crianças** (`/admin/children`).

Exercícios

- ☐ Montar um evento de teste com 2 times e 3 checkpoints posicionados no mapa.
- ☐ Criar um usuário de recepção e entrar com ele.
- ☐ Conferir no telão (`/display/map`) que os checkpoints aparecem nos lugares certos.
- ☐ Exportar ou consultar o relatório do evento de teste.

Erros comuns: esquecer de posicionar checkpoints no mapa; `CHECKPOINT_ID` do aparelho diferente do cadastro; usuário com perfil errado.

Trilha 4 · Famílias (roteiro para a recepção)

1. O responsável recebe o link de convite (`/family/invite/<token>`) e cria a conta.

2. A recepção aprova o vínculo; o acesso só funciona depois disso.
3. No app Pulyn Family, o responsável entra com e-mail e senha, toca em vincular e lê o QR Code da criança.
4. Antes da festa o app mostra a pré-festa; com o evento ativo abre o mapa ao vivo; ao final mostra o resultado.
5. Na aba de perfil, o responsável gerencia as crianças vinculadas.

Perguntas frequentes: "Não vejo meu filho" (vínculo ainda pendente ou QR de outro evento); "O avatar não mexe" (criança ainda não passou em nenhum checkpoint, ou checkpoint sem posição no mapa).

Trilha 5 • Técnica

Objetivo: instalar, publicar e diagnosticar o sistema.

1. Subir o backend local: `npm install`, `.env` a partir de `.env.example`, `npm run dev`, `GET /api/test`.
2. Subir o painel (`npm run dev` em `front-pulyn`) e o app (`flutter pub get`, `flutter run`).
3. Trocar o ambiente da API: no app, `ApiConfig.currentEnvironment` ou a tela de seleção de ambiente; no ESP32, `PULYN_LAN_MODE`.
4. Gravar um checkpoint: Arduino IDE, placa ESP32 Dev Module, pasta `pullynC2_modular`, monitor serial em 115200.
5. Seguir a árvore de diagnóstico de um avatar que não se move (seção de rastreo) e ler os logs com prefixo `[RASTREIO]`, `[MAP]`, `[TRACKING]` e `[WebSocket]`.
6. Rodar os testes (`npm test`, `flutter test`, `npm run typecheck`) e ler a lista de problemas conhecidos.

Exercícios

- ☐ Provocar um checkpoint sem coordenadas e achar a mensagem de erro no log do backend.
- ☐ Conectar um cliente WebSocket com e sem token e comparar o resultado.
- ☐ Trocar o ESP32 do servidor online para o local e ler uma pulseira.
- ☐ Corrigir um dos itens de prioridade baixa da lista de problemas, com teste.

Avaliação final

A equipe simula uma festa de 15 minutos: a recepção cadastra 4 crianças, o admin já deixou o evento pronto, o game master conduz uma partida e um responsável acompanha pelo app. O treinamento é considerado concluído quando, sem ajuda, cada pessoa

completa as tarefas do seu perfil e a equipe resolve um problema sorteado: pulseira já vinculada, checkpoint offline ou avatar parado.